



## **SVM**

Introdução ao Reconhecimento de Padrões  
Turma TB

**Gabriel Vaz Cançado Ferreira**

Matrícula: 2021015194

## 1. Gerar dados em espiral

Como o pacote mlbench não está disponível em python, criamos uma função para gerar dados em espiral.

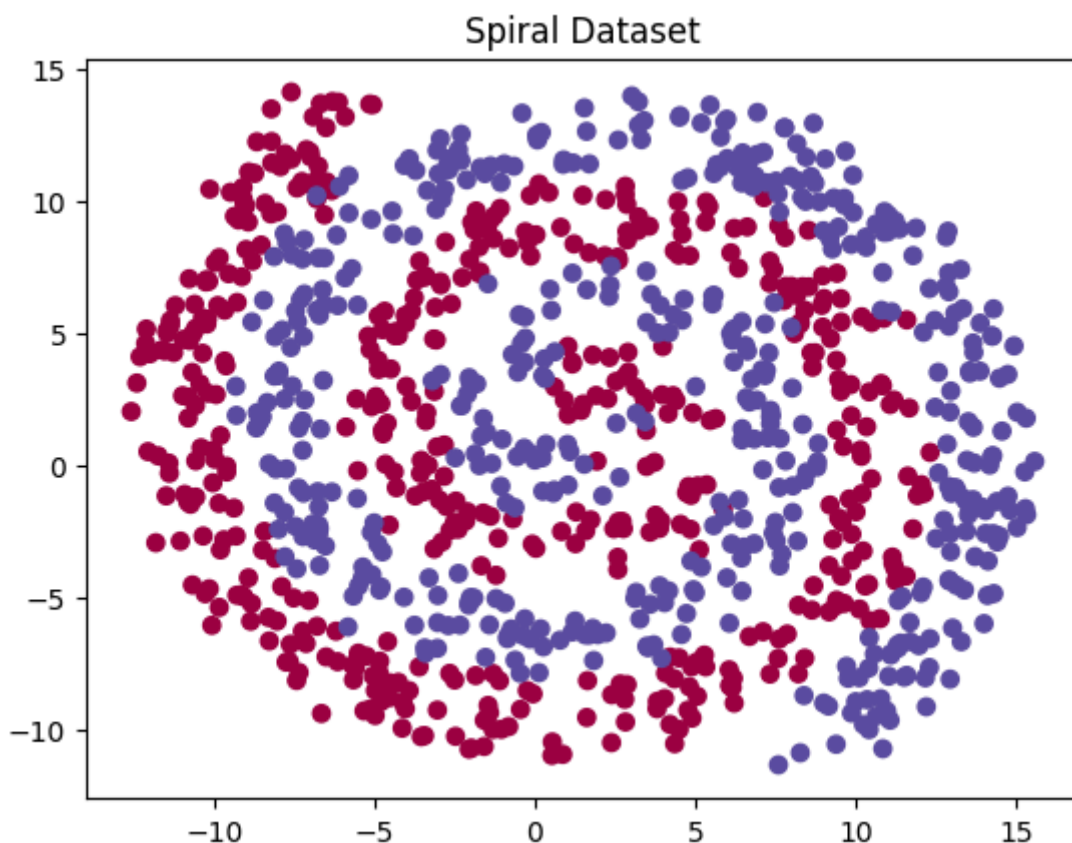


```
def generate_spiral(n_points, noise=0.001):
    n = np.sqrt(np.random.rand(n_points)) * 780 * (2 * np.pi) / 360
    d1x = -np.cos(n) * n + np.random.rand(n_points) * noise
    d1y = np.sin(n) * n + np.random.rand(n_points) * noise

    d2x = np.cos(n) * n + np.random.rand(n_points) * noise
    d2y = -np.sin(n) * n + np.random.rand(n_points) * noise

    X = np.concatenate((np.vstack((d1x, d1y)).T, np.vstack((d2x, d2y)).T))
    y = np.concatenate((np.zeros(n_points), np.ones(n_points)))

    return X, y
```



## 2. Separação dos dados e treinamento do modelo

Para separar os dados, usamos a função `train_test_split` do sklearn. Para o modelo de SVM, usamos a função `SVC` disponível no pacote `sklearn.svm`.



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = SVC(kernel='rbf', C=1, gamma=0.1)

clf.fit(X_train, y_train)

x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
                     np.linspace(y_min, y_max, 500))

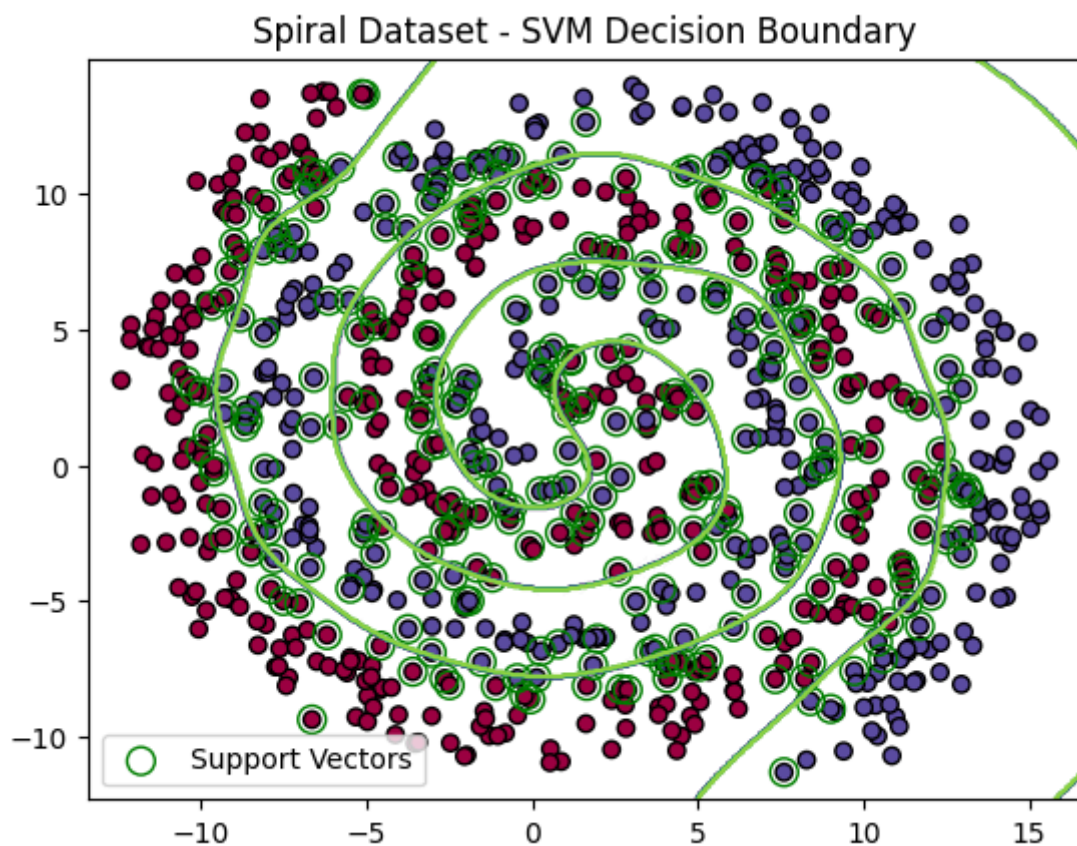
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contour(xx, yy, Z, alpha=0.8, c = 'black')

plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=plt.cm.Spectral, edgecolors='k')

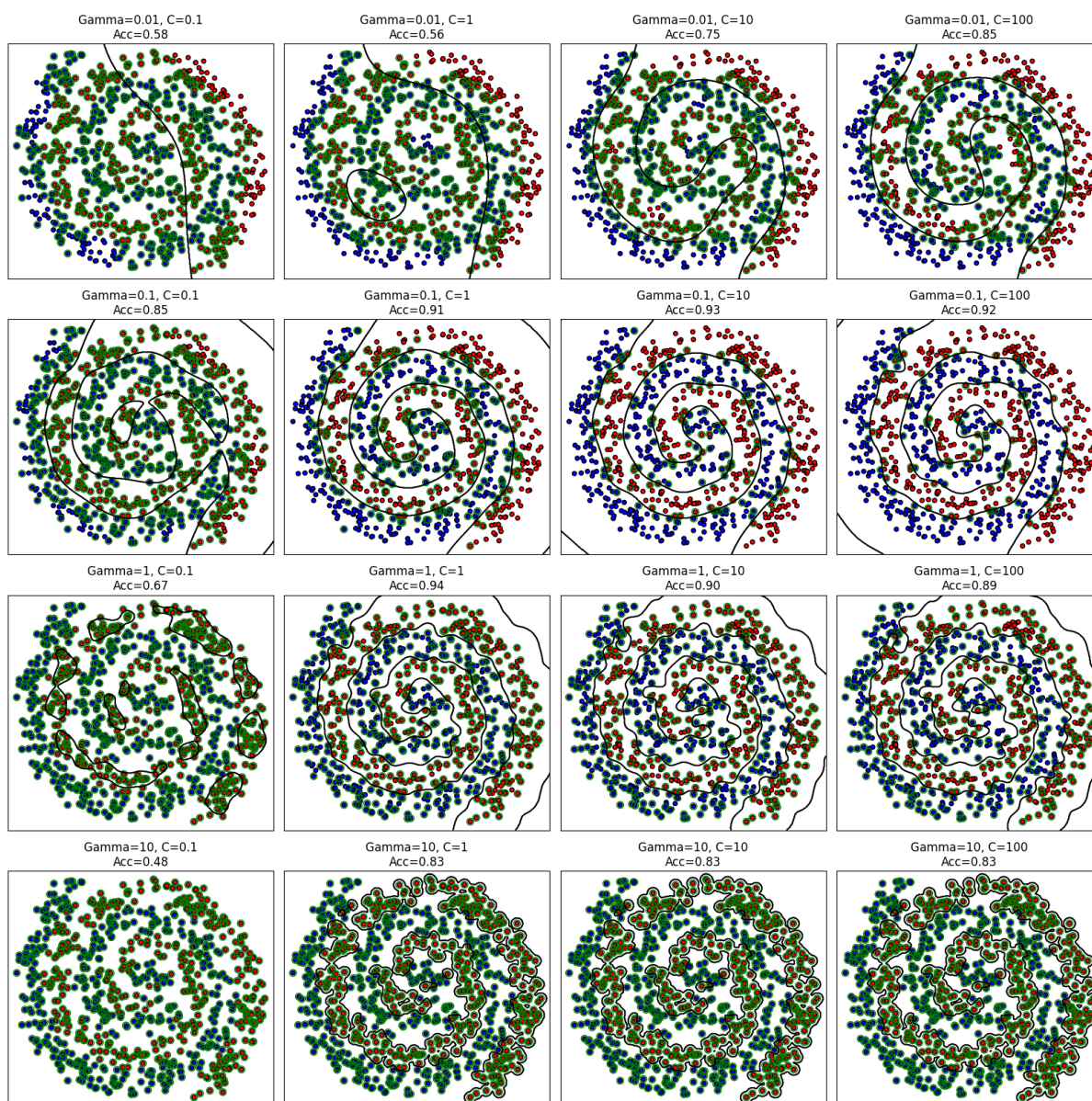
plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
            facecolors='none', edgecolors='green', s=100, label='Support Vectors')

plt.title("Spiral Dataset - SVM Decision Boundary")
plt.legend()
plt.show()
```



### 3. Variação dos parâmetros

Para compreender o efeito dos parâmetros na acurácia do modelo, testamos a acurácia para diferentes valores de  $C$  e  $\gamma$ . O  $C$  é o parâmetro de regularização das funções de penalidade utilizadas no processo de otimização das SVM. O  $\gamma$  por sua vez define o espalhamento dos vetores de suporte nos dados escolhidos.



Foram avaliadas combinações de parâmetros do classificador SVM com kernel RBF variando-se o parâmetro de suavidade do kernel ( $\gamma$ ) e o parâmetro de regularização ( $C$ ). A mudança desses hiperparâmetros mostrou impacto claro tanto na forma da fronteira de decisão quanto na acurácia observada.



Impacto do  $\gamma$  (gamma):

Valores muito baixos de  $\gamma$  (ex.: 0.01) produzem fronteiras excessivamente suaves, incapazes de seguir a estrutura espiral do dataset, caracterizando underfitting. Conforme  $\gamma$  aumenta até cerca de 0.1–1, o modelo consegue capturar adequadamente o formato dos dados, com contornos que seguem as espirais de maneira consistente, resultando nas maiores acurácias do grid. Acima disso ( $\gamma=10$ ), o modelo passa a se ajustar demais aos pontos de treinamento, com fronteiras altamente tortuosas, indicando overfitting.

Impacto do C:

Valores baixos de C (ex.: 0.1) produzem margens mais largas e permitem erros no treinamento, levando a uma fronteira simplificada que frequentemente subajusta os dados. À medida que C aumenta, o modelo torna-se menos tolerante a erros e segue mais de perto as amostras, melhorando o encaixe nas regiões onde o padrão é bem definido. Contudo, valores muito altos (ex.: C=100) não trazem ganhos significativos adicionais e tendem a aumentar a sensibilidade a ruído, piorando a generalização.

Região de melhor desempenho:

Os melhores resultados visuais e numéricos surgem tipicamente na faixa intermediária — aproximadamente entre  $\gamma \in [0.1, 1]$  e  $C \in [1, 10]$ . Nessa região, a fronteira acompanha a geometria da espiral sem reproduzir variações espúrias do ruído, alcançando acurácias na faixa de 0.90–0.94. Já as regiões extremas do grid evidenciam claramente underfitting ( $\gamma$  muito baixo e C baixo) ou overfitting ( $\gamma$  alto e C alto).

Conclusão:

O modelo SVM com kernel RBF apresenta forte sensibilidade a  $\gamma$  e C no contexto de dados não-lineares como a espiral. Os resultados confirmam a necessidade de calibrar  $\gamma$  para permitir curvatura suficiente do kernel e de ajustar C de modo a balancear margem ampla com penalização adequada a erros. Na prática, pares como ( $\gamma=0.1$ , C=10) ou ( $\gamma=1$ , C=1) configuram pontos de equilíbrio eficazes, combinando boa definição de fronteiras com alto desempenho de classificação.